

# Module 12 - chapter 3

## Providers in Terraform

<https://registry.terraform.io/browse/providers>

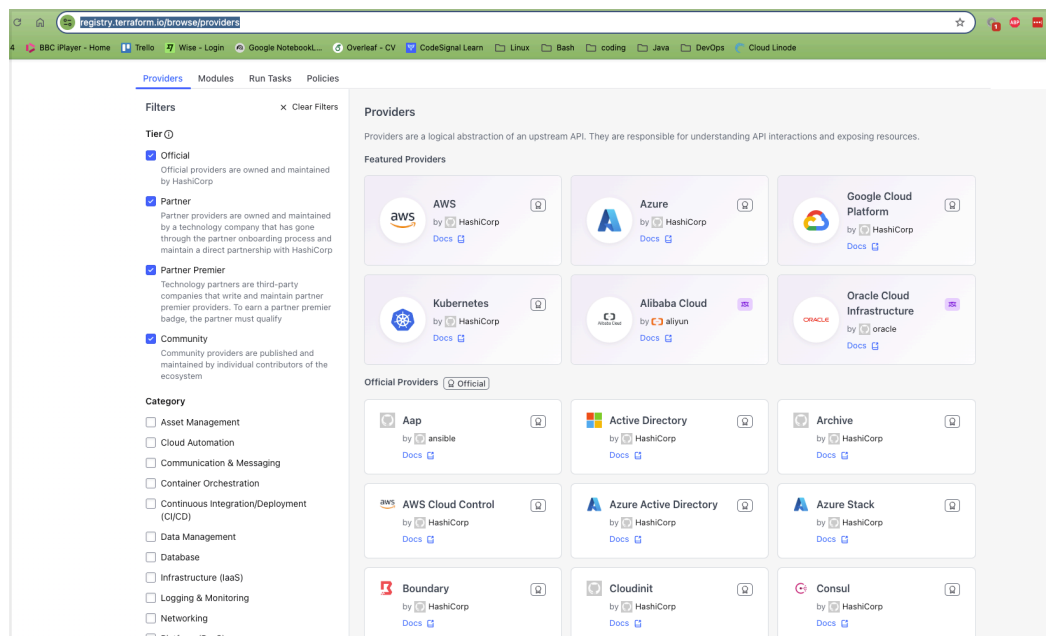
We need to connect to our AWS account.

Provider -> expose resources for specific infrastructure platform like AWS

Provider -> responsible for understanding the API of that platform

Provider -> is just code that knows how to talk to specific technology or platform

In terraform providers web page we can see a list of providers that terraform supports:



For example AWS provider, takes the terraform configuration and translates it to something that AWS API understands.

If we select AWS -> we can get to its documentation

**Terraform Registry** | Search all resources

Providers / hashicorp / aws / Version 6.9.0 / Latest Version

**aws** 

Overview Documentation **USE PROVIDER**

### AWS Provider

The Amazon Web Services (AWS) provider is Terraform's most widely-used provider and the industry-standard way to manage AWS infrastructure as code. It is an indispensable part of how leading technology companies, global banks, government agencies, and some of the largest enterprises in the world build and operate in the cloud. Every day, it provisions and orchestrates billions of dollars of AWS infrastructure across thousands of organizations.

With 1,514 resources and 609 data sources, the AWS provider spans the full breadth of AWS services—from foundational capabilities like compute, storage, networking, and identity management to advanced services for AI, analytics, and event-driven architectures, including Lambda, RDS, SageMaker, and Bedrock. Whether you're provisioning a single S3 bucket or orchestrating a multi-region, multi-account, multi-tenant environment, the AWS provider delivers consistent, reliable workflows that scale with your needs.

Configure the provider with your AWS credentials, and you can immediately begin creating and managing infrastructure in a safe, repeatable way. Use the navigation on the left to explore the available resources, or start with our [Get Started tutorials](#) to learn the fundamentals. For deeper guidance on specific AWS services, visit the [AWS services tutorials](#).

**Multi-language provider docs**  
Terraform

The Registry now supports multi-language docs powered by CDK for Terraform. [Learn more](#)

ON THIS PAGE

- Example Usage
- Authentication and Configuration
- AWS Configuration Reference
- Custom User-Agent information
- Argument Reference
- Getting the Account ID

**aws provider**

- > Guides
- > Functions
- > ACM (Certificate Manager)
- > ACM PCA (Certificate Manager Private Certificate Authority)
- > AMP (Managed Prometheus)
- > API Gateway
- > API Gateway V2
- > Account Management
- > Amazon Q Business
- > Amplify
- > App Mesh
- > App Runner

**well documented!**

**TECHNOLOGY WITH A PURPOSE**

Easy to find on a search engine:

**Google**

All Images Videos Short videos Forums Web News More

**Terraform Registry**  
https://registry.terraform.io/docs/resources/instance

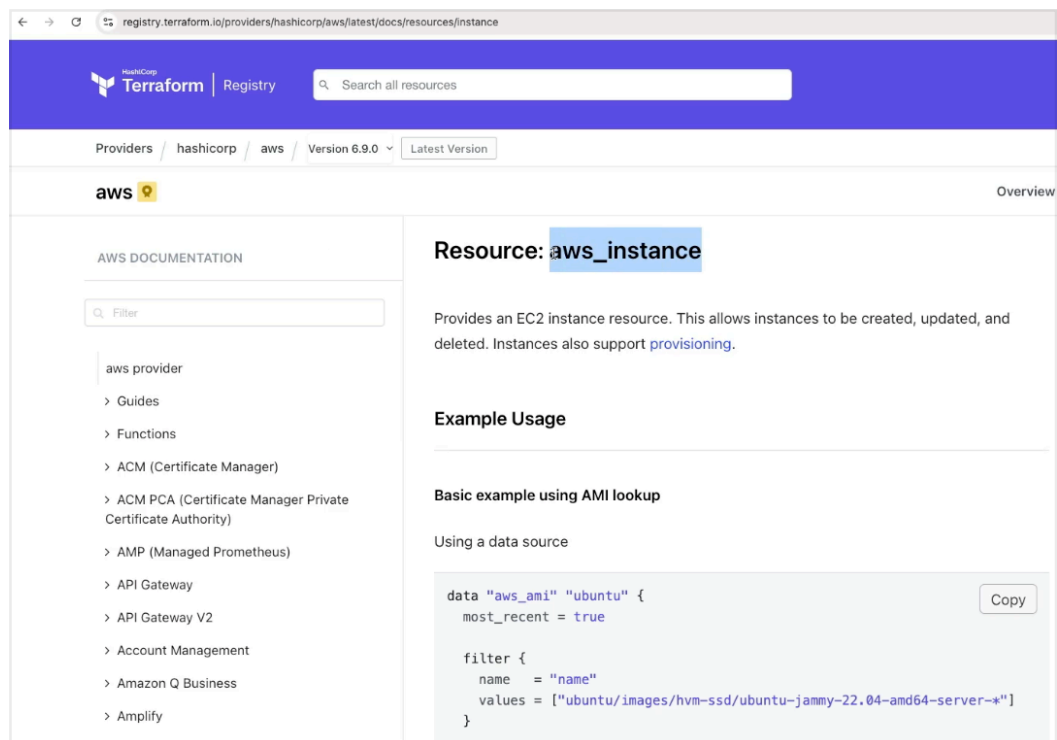
[aws\\_instance](#) | [Resources](#) | [hashicorp/aws - Terraform Registry](#)

[Aws\\_ec2\\_instance\\_state](#) [Aws\\_ami\\_from\\_instance](#) [Aws\\_spot\\_instance\\_request](#)

**Terraform Registry**  
https://registry.terraform.io/ec2-instance/aws/latest

**terraform-aws-modules - ec2-instance**

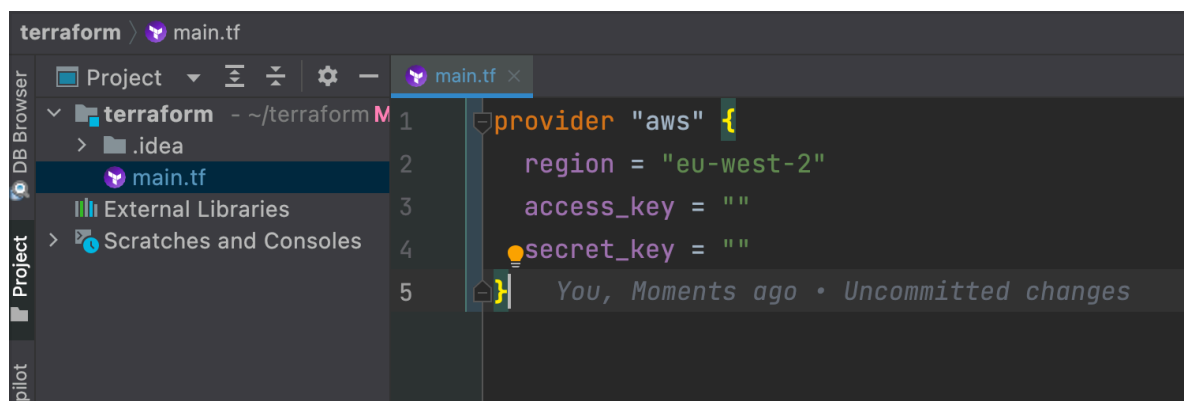
1 Jul 2025 — Terraform module which creates an EC2 instance on AWS. SWUbanner. Usage. Single

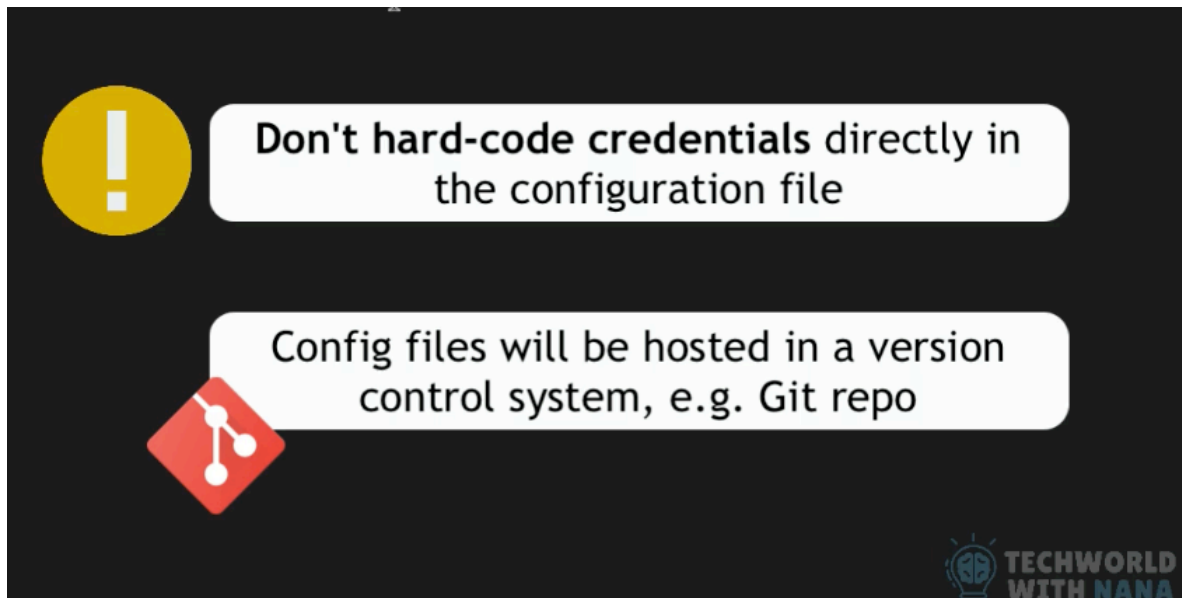


## Install and connect to Provider

Let's use the AWS provider in our empty file main.tf file and we will pass the **provider configuration** that will allow TF to connect to our AWS account:

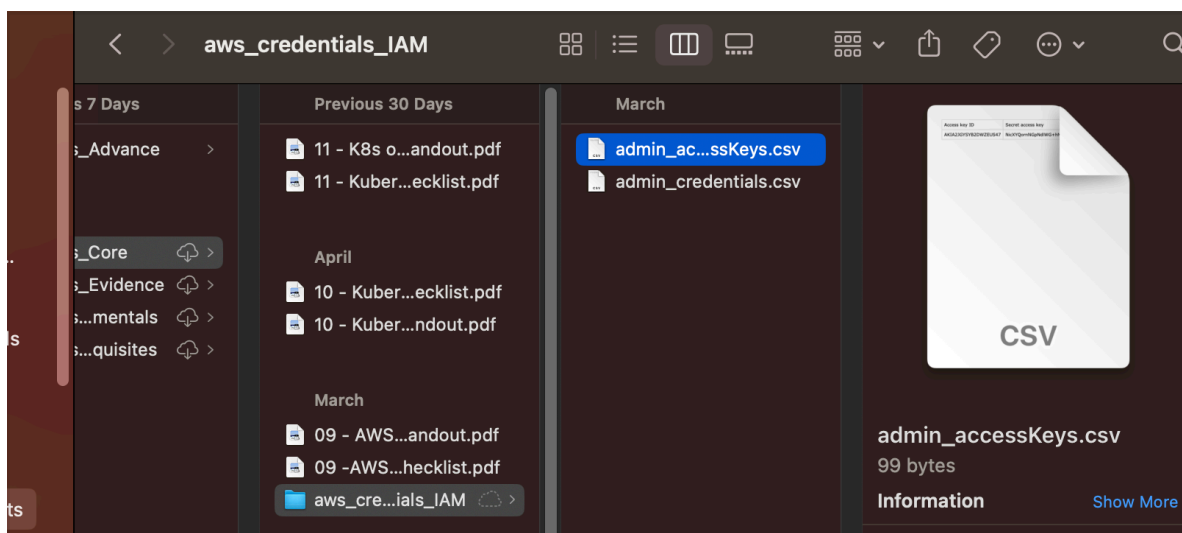
```
provider "aws" {  
  region = "eu-west-2"  
  access_key = ""  
  secret_key = ""  
}
```

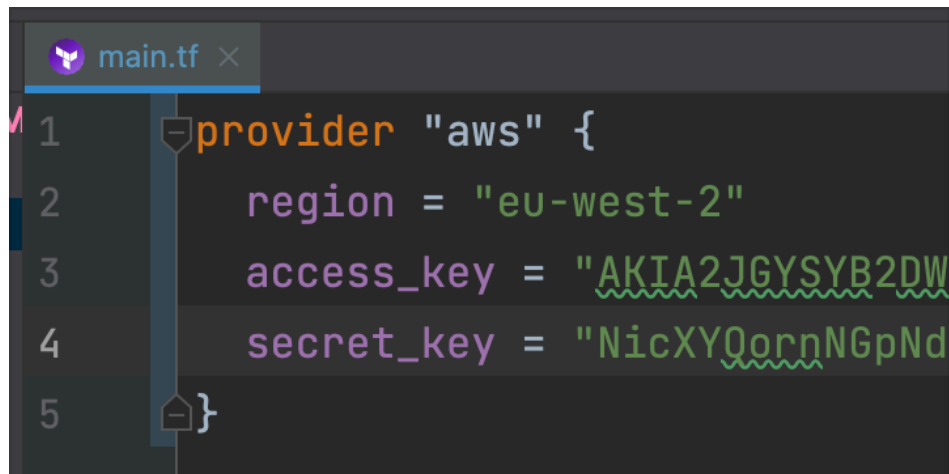




Hardcoding sensitive data in code is not good practice, we use technologies like GitHub to host the code and many ppl can see the sensitive data.

But for now we will hard-code the credentials, we will use the access key and secret key of the 'admin' user that we created in our AWS account. I downloaded the 'admin' access and secret keys as a .csv file so I can get those values from there:





```
1 provider "aws" {  
2     region = "eu-west-2"  
3     access_key = "AKIA2JGYSYB2DW"  
4     secret_key = "NicXYQornNGpNd"  
5 }
```

How do we know which attributes we need? -> documentation

AWS -> documentation -> Authentication and Configuration -> usage

### Provider Configuration

 **Warning:**  
Hard-coded credentials are not recommended in any Terraform configuration and risks secret leakage should this file ever be committed to a public version control system.

Credentials can be provided by adding an `access_key` , `secret_key` , and optionally `token` , to the `aws` provider block.

Usage:

```
provider "aws" {  
    region      = "us-west-2"  
    access_key  = "my-access-key"  
    secret_key  = "my-secret-key"  
}
```

Copy

When we installed TF the providers didn't get installed automatically because there are so many providers and we don't need all of them. So we install what we actually need when need it instead.

So when we define a provider configuration we also need to make sure we install it.

# Providers need to be installed

Like we do with dependencies when coding in java for example.

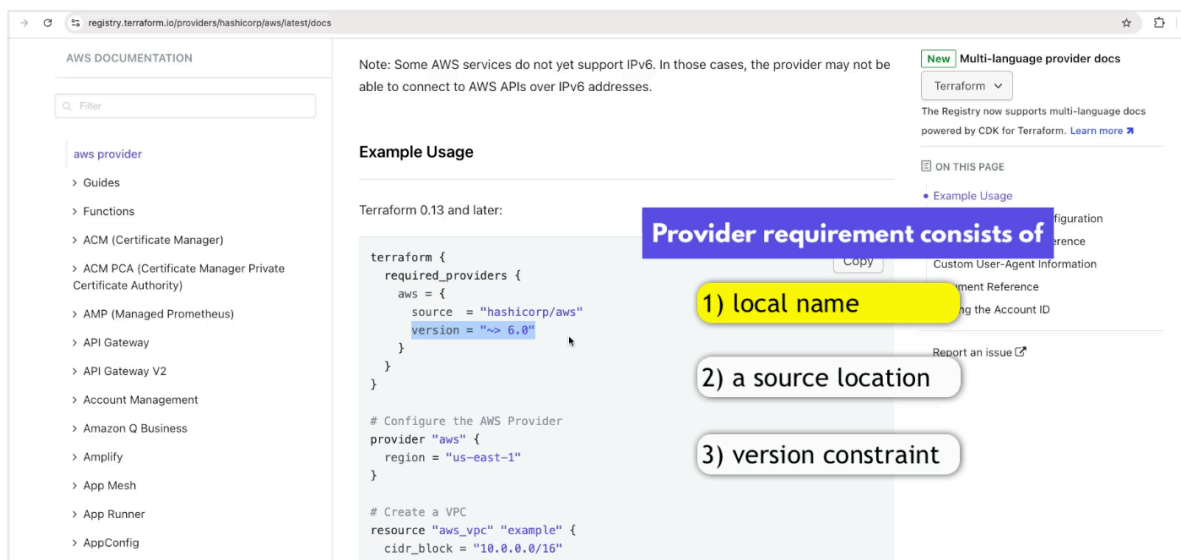
We need to define the provider with this code, where we get the latest version of the provider:



```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

# Configure the AWS Provider
provider "aws" {
  region = "us-east-1"
}
```

We don't have to have it in the case of AWS, but is good practice to have it explicitly.



Provider requirement consists of

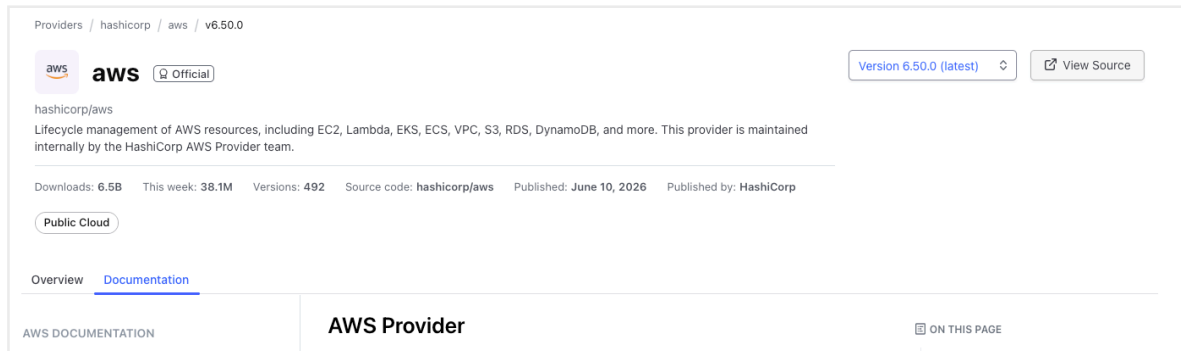
- 1) local name
- 2) a source location
- 3) version constraint

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

👉 get latest major version

}

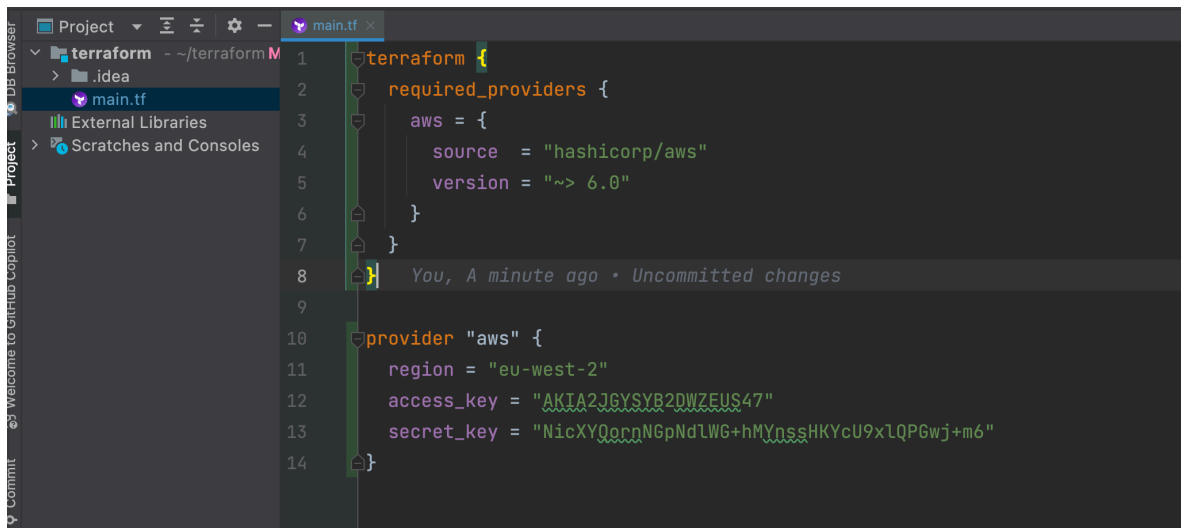
Or we can hardcode the latest version using this value



The screenshot shows the HashiCorp Terraform Registry page for the `aws` provider. The page header indicates the provider is `hashicorp/aws` at version `v6.50.0`. The provider is marked as 'Official'. A dropdown menu shows 'Version 6.50.0 (latest)' and a 'View Source' button is present. The description states: 'hashicorp/aws Lifecycle management of AWS resources, including EC2, Lambda, EKS, ECS, VPC, S3, RDS, DynamoDB, and more. This provider is maintained internally by the HashiCorp AWS Provider team.' Statistics show 6.5B downloads, 38.1M this week, and 492 versions. The source code is `hashicorp/aws`, published on June 10, 2026, by HashiCorp. A 'Public Cloud' badge is visible. The page has tabs for 'Overview' and 'Documentation', with 'Documentation' being the active tab. The main content area is titled 'AWS Provider' and includes a 'ON THIS PAGE' section.

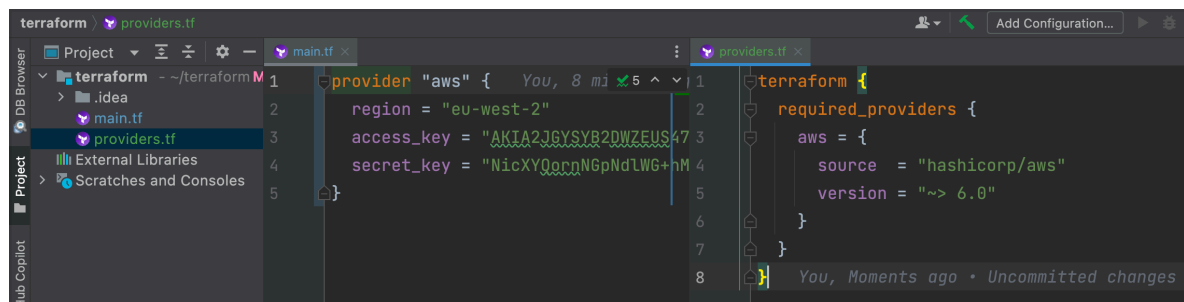
```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "6.50.0"      👉 get hardcoded specific latest version  
    }  
  }  
}
```

So I add the provider as a dependency declaration - it tells Terraform "I need the AWS provider, version 6.x."



The screenshot shows an IDE with two files open. The first file, `main.tf`, contains the `terraform` block with `required_providers` configuration for the `aws` provider, specifying `source = "hashicorp/aws"` and `version = "~> 6.0"`. The second file, `provider.tf`, contains the `provider "aws" {` block with configuration for the AWS provider, including `region = "eu-west-2"`, `access_key = "AKIA2JGYSYB2DWZEUS47"`, and `secret_key = "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"`. The IDE interface includes a project browser on the left showing the file structure and a status bar at the bottom.

And I also can have a dedicated file to have the dependencies declarations -> `providers.tf` (I can name it however I want)

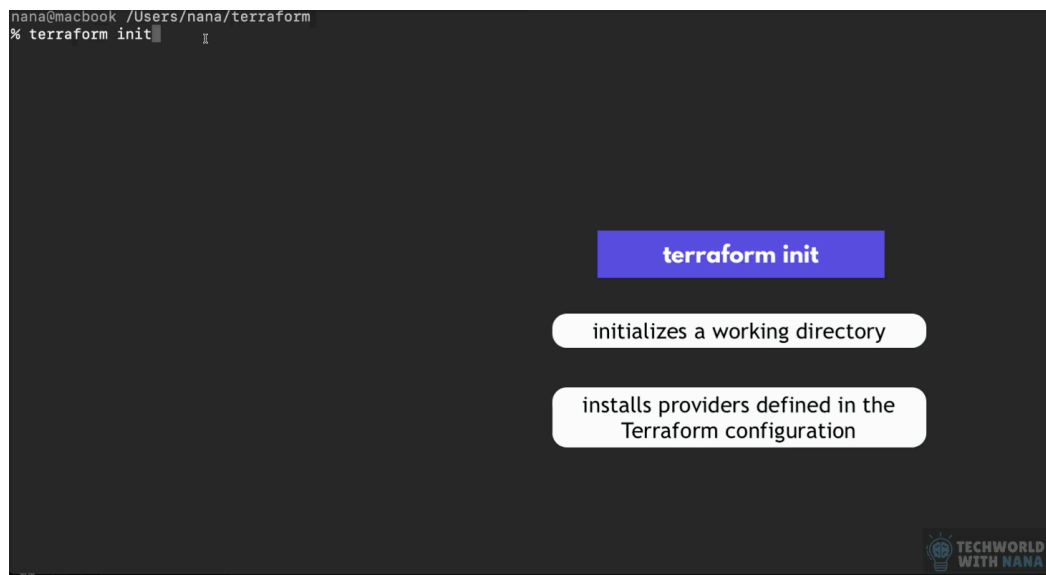


```
1 provider "aws" {
2   region = "eu-west-2"
3   access_key = "AKIA2JGYSYB2DWZEUS47"
4   secret_key = "NicXYQppnNGpNdLWG+hM"
5 }

1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 6.0"
6     }
7   }
8 }
```

Now we can install the provider:

-> terraform init



Q:

I can have the dependencies declarations in a different .tf file and the file can be named however we want. so when I execute terrafrom init how does terrafrom know where the dependencies are?

A:

Terraform reads all .tf files in the current directory automatically — it doesn't matter what the files are named. It merges them all together before doing anything.

So when you run terraform init, Terraform scans every .tf file in the directory, finds all terraform {} blocks, and installs the required providers.

That's why file naming is flexible — main.tf, providers.tf, dependencies.tf — Terraform doesn't care, it reads them all.



```

~ cd terraform
terraform git:(Module_12/Terraform_chapter-3) x terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.50.0...
- Installed hashicorp/aws v6.50.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

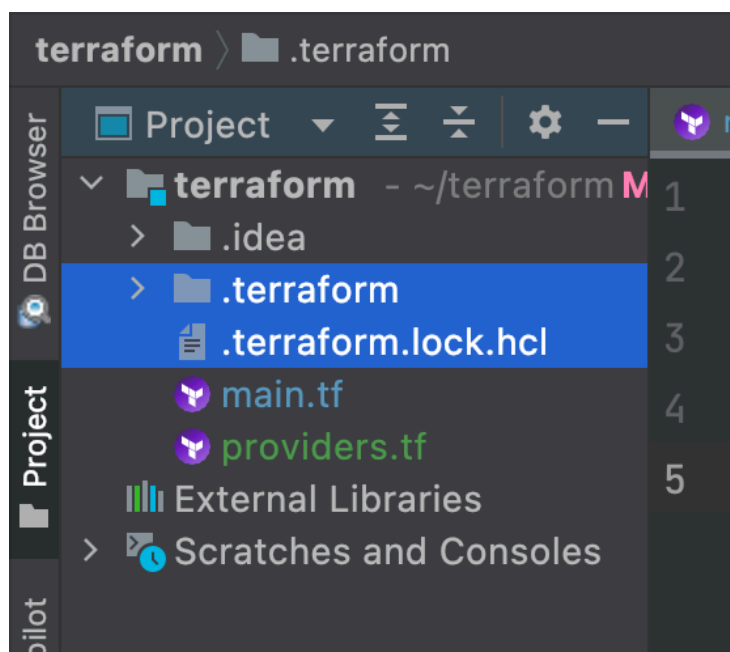
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
terraform git:(Module_12/Terraform_chapter-3) x

```

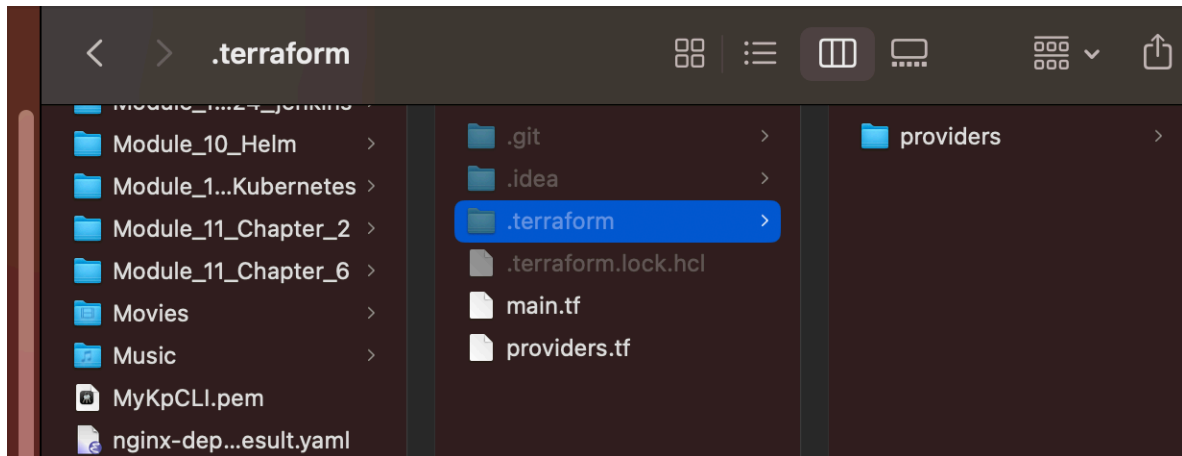
Terraform finds the latest version of the major version 6.0 which is 6.50 🙌

And AWS provider is successfully installed and terraform has been successfully initialised!

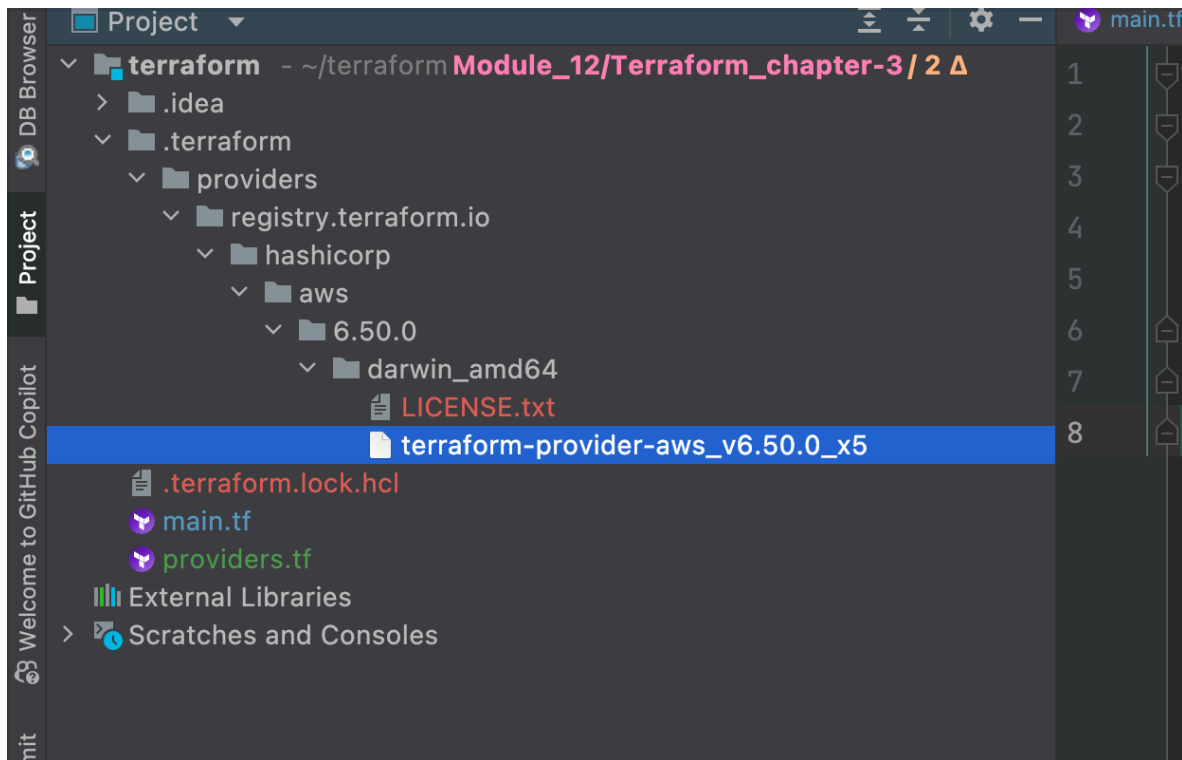
And we can see our 'terraform' folder now has a new folder and another file:



To show hidden files in Finder press -> Cmd + Shift + .



This file is the downloaded code that will talk to AWS for us 'terraform-provider-aws\_v6.50.0\_x5', this is the AWS provider code:



And this file '`.terraform.lock.hcl`' keeps track of the providers we have downloaded:

```

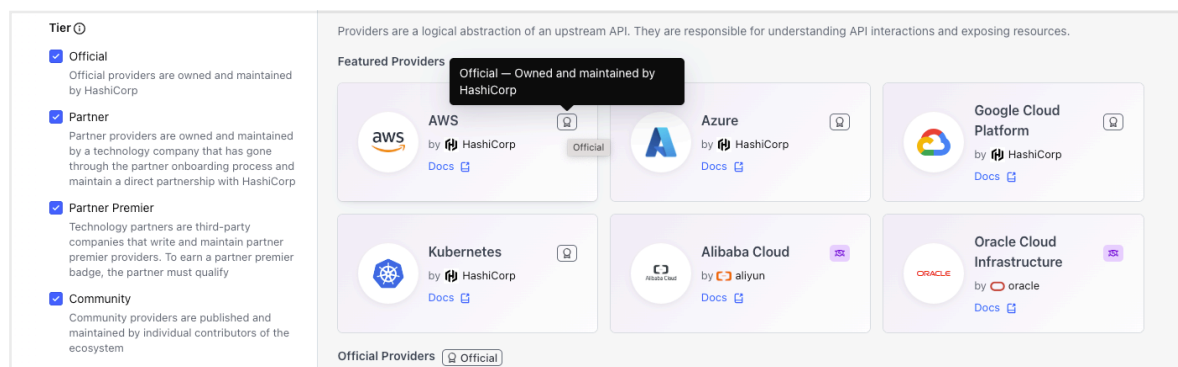
1  # This file is maintained automatically by "terraform init".
2  # Manual edits may be lost in future updates.
3
4  provider "registry.terraform.io/hashicorp/aws" {
5      version = "6.50.0"
6      constraints = ["~> 6.0"]
7
8      hashes = [
9          "h1:Uf2L1f5ba8dksEUk0oiQbEbkiG0R6tUE/0tCd36Xzk=",
10         "zh:0072806bb262c6d86bc25b4e75750e469881144c14818afdb8a7b82db840e1588",
11         "zh:1ebc2dae335dad7a8b16a1985b69a63a14954282bb44fdba7d5103f77551ac7b",
12         "zh:2dab48fe8f3193b8216d578ac1e3674fa566435cc7dbce2953d55b72e31d0241",
13         "zh:2fc3d3029c2b7429472391ef339672e1fca8e6ff32c8a519bf3accedafa7e24fe",
14         "zh:38a36e64e7212f6c8d861ea4d449cce07131b3378de601bf9d49a99e000208",
15         "zh:3ac70758ed251ce78b7f541a5a79cc6fe56474412783ae1d8ccaf719bdd0f30bf",
16         "zh:4385d3903e685b9d9b2b8005b4eb7db89f030267d4d03c7d792d2f5e739cc874a",
17         "zh:4c0e0760b87bfaf51f30faag2a737f4183b7c615f4a86557f7d3c893a610dc5",
18         "zh:4f8a8ed18694239b896c6415d9a1e5ef89e1da4f4ad60924aa0522ad8b1f6599",
19         "zh:502fca2be1c95f443c3e67d0555601d1de65b4ca82d197c059e9c86830e3a0a",
20         "zh:57d037f6fd045f2660909c3bdfac9622d81165ce647479cba98d1f353c5eab",
21         "zh:5dc5a0b915c2ac5256d909458f5c8e40b35f78b3a36ea893c86624eaf6c54e37",
22         "zh:9b12af85486a96aged8d7984b0ff811a4b42e3d88dad1a3fb4c0b580d04fa425",
23         "zh:b84c87c58a320agdb2c74a4cad03ae5aac7f2eae21db26f00fdde98c8c4d4523",
24         "zh:c895f1d5c8cbeff77850ac99efd36bde0048d4e909b296882331b9b9ebf48cfa",
25         "zh:ead82831683619124597a1f170dd31e9b293e9cf22f558cb166d5e734fcd11e4",
26     ]
27 }

```

So if we add another provider -> it will appear in this file 🙌

We mentioned before that We don't have to have the dependency declaration code in the case of AWS, but is good practice to have it explicitly.

Why AWS doesn't need the dependency declaration? Because is part of hashicorp registry, so it is 'official'



For providers that are not 'official' hence not in the terraform registry they have to be added to the code otherwise terraform won't know where to get the provider from as it looks at the attribute 'source' to know where to go.

So if the provider belongs to Tf registry terraform knows the location of the provider -> 'HashiCorp/<provider\_name>'

Example -> Linode which is not 'official' it is a 'verified' provider

It has the source as

-> source = "linode/linode"

Overview [Documentation](#)

LINODE DOCUMENTATION

linode provider

- > Guides
- > Resources
- > Data Sources

## Linode Provider

[Report an issue](#)

The Linode provider exposes resources and data sources to interact with [Linode](#) services. The provider needs to be configured with the proper credentials before it can be used.

Use the navigation to the left to read about the available data sources.

### Example Usage

Terraform 0.13 and later:

```
terraform {
  required_providers {
    linode = {
      source = "linode/linode"
      # version = "..."
```

If we don't declare the dependency, TF won't know where to get it from.

So let's only add the code for the Linode provider configuration without the dependency:

```
terraform > main.tf
Project
  terraform - ~/terraform Module_12/
    .idea
    .terraform
    .terraform.lock.hcl
    main.tf
    providers.tf
    External Libraries
    Scratches and Consoles
  DB Browser
  Project
  welcome to GitHub Copilot

1 provider "aws" {
2   region = "eu-west-2"
3   access_key = "AKIA2JGYSYB2DWZEUS47"
4   secret_key = "NicXYQornNGpNdLWG+hMYnssHKYcU9xLQPGwj+m"
5 }
6
7 provider "linode" {
8   # token = "..."
9 }
```

-> terraform init

```
+ terraform git:(Module_12/Terraform_chapter-3) x terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/linode...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Error: Failed to query available provider packages

Could not retrieve the list of available versions for provider hashicorp/linode: provider registry registry.terraform.io does not have a provider named registry.terraform.io/hashicorp/linode

Did you intend to use linode/linode? If so, you must specify that source address in each module which requires that provider. To see which modules are currently depending on hashicorp/linode, run the following command:
    terraform providers

+ terraform git:(Module_12/Terraform_chapter-3) x
```

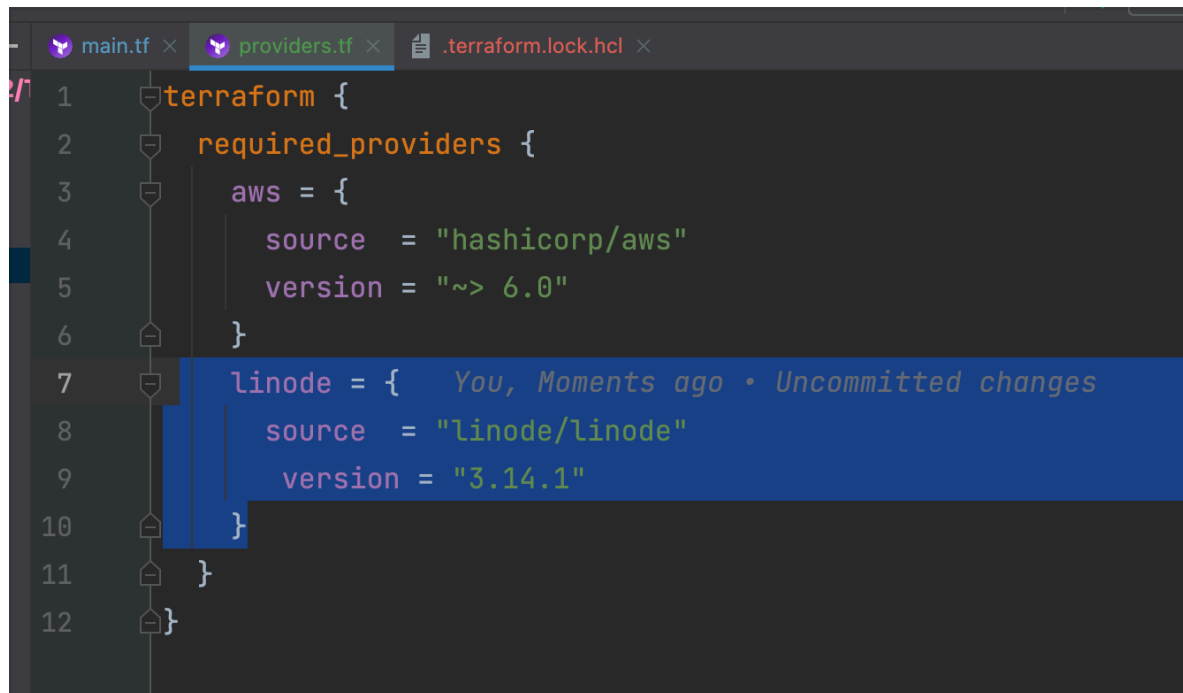
We got error -> Failed to query available provider packages

And we can see that it tried to find the provider in HashiCorp registry->  
hashicorp/linode 🙅

And we know Linode is not in the terraform registry, it is a third party provider.

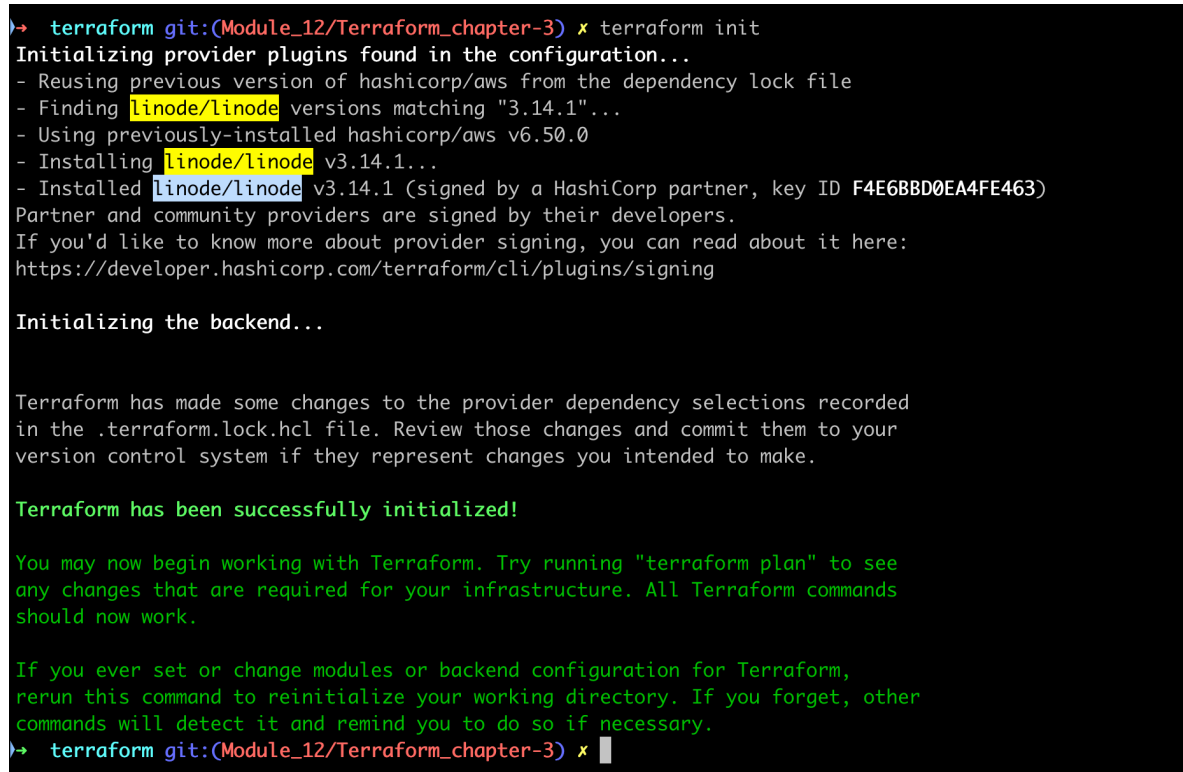
And it also says "does not have a provider named `registry.terraform.io/hashicorp/linode`" 🙅

I need to add Linode dependency declaration:



```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 6.0"
6     }
7     linode = {
8       source = "linode/linode"
9       version = "3.14.1"
10    }
11  }
12 }
```

-> terraform init



```
> terraform git:(Module_12/Terraform_chapter-3) x terraform init
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Finding linode/linode versions matching "3.14.1"...
- Using previously-installed hashicorp/aws v6.50.0
- Installing linode/linode v3.14.1...
- Installed linode/linode v3.14.1 (signed by a HashiCorp partner, key ID F4E6BBD0EA4FE463)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing

Initializing the backend...

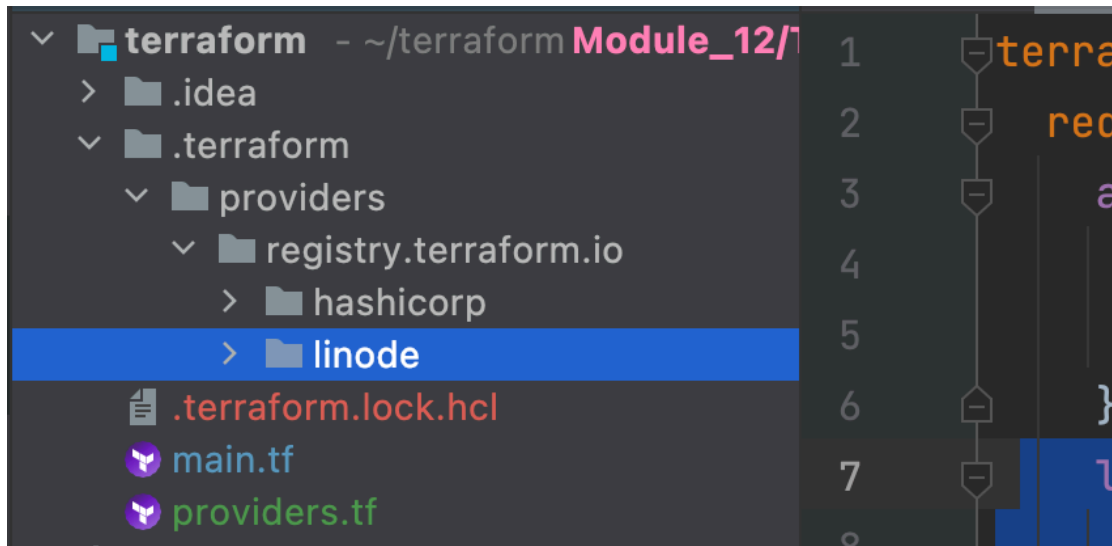
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
> terraform git:(Module_12/Terraform_chapter-3) x
```

It works! And now I have Linode provider code available locally:



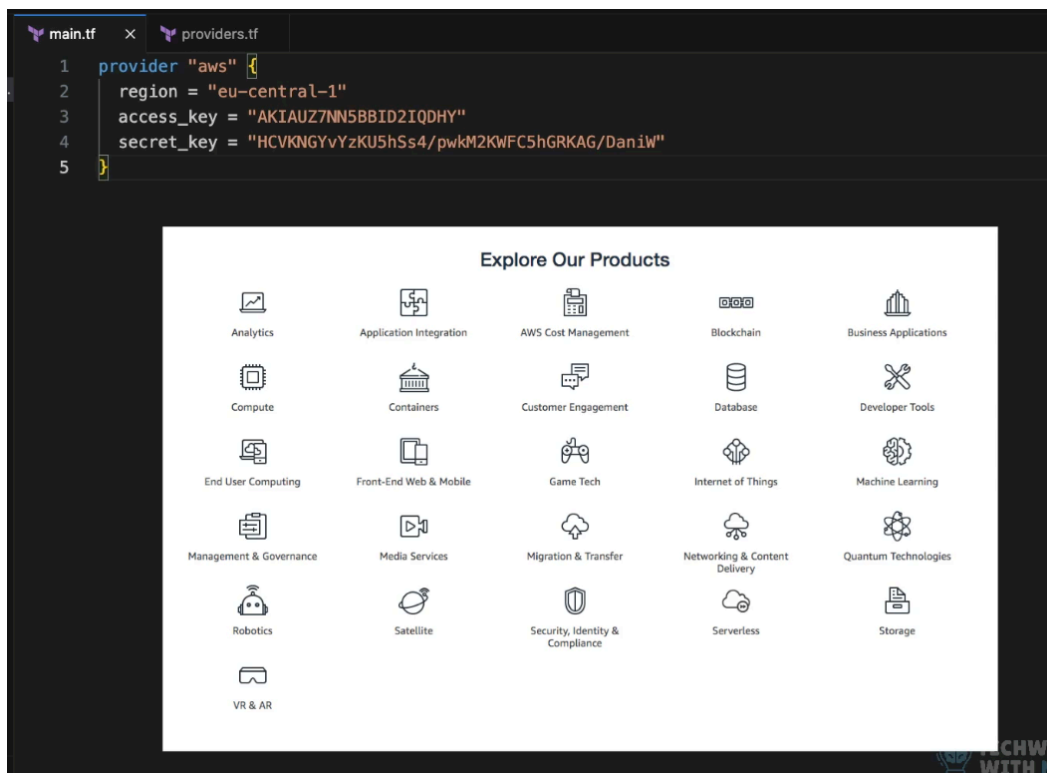
Dependency declaration:

- > optional for 'official' HashiCorp providers
- > mandatory for 'non official' providers

## Providers exposing resources

Now we have a provider -> we can use the resources it provides

The provider gives us access to the entire API in our case the entire API of AWS



The provider give us access to every single service in AWS and to the resources of those services.

And in the AWS provider documentation we can see the services that are exposed:

OverviewDocumentation

AWS DOCUMENTATION

> DocumentDB

> DocumentDB Elastic

> DynamoDB

> DynamoDB Accelerator (DAX)

> EBS (EC2)

> EC2 (Elastic Compute Cloud)

> Actions

aws\_ec2\_stop\_instance

> Resources

aws\_ami

aws\_ami\_copy

aws\_ami\_from\_instance

aws\_ami\_launch\_permission

aws\_ec2\_allowed\_images\_settings

aws\_ec2\_availability\_zone\_group

aws\_ec2\_capacity\_block\_reservation

aws\_ec2\_capacity\_reservation

aws\_ec2\_default\_credit\_specification

aws\_ec2\_fleet

aws\_ec2\_host

aws\_ec2\_image\_block\_public

## AWS Provider

The Amazon Web Services (AWS) provider is Terraform's the industry-standard way to manage AWS infrastructure. It's part of how leading technology companies, global enterprises, and the largest enterprises in the world build and operate their cloud provisions and orchestrates billions of dollars of AWS organizations.

With 1,667 resources, 10 ephemeral resources, 14 data sources, the AWS provider spans the full breadth of AWS services like compute, storage, networking, and identity management, analytics, and event-driven architectures, including Amazon Bedrock. Whether automating a single S3 bucket in an enterprise-scale environment, the provider delivers infrastructure at scale with your needs.

Configure the provider with your AWS credentials and manage infrastructure in a safe, repeatable way. You can explore the available resources, or start with our [AWS Provider fundamentals](#). For deeper guidance on specific AWS services, see [AWS Provider tutorials](#).

Note: Some AWS services do not yet support IPv6. Terraform is unable to connect to AWS APIs over IPv6 addresses.

### Example Usage

Terraform 0.13 and later: